

PONTIFICIA UNIVERSIDAD CATÓLICA DEL PERÚ
FACULTAD DE CIENCIAS E INGENIERÍA

1INF29 SISTEMAS OPERATIVOS

Semestre 2025-1

Laboratorio 2

1) (10 puntos) A ChatGPT (<https://chatgpt.com/>) se le envió el siguiente requerimiento:

El siguiente código genera una cadena de procesos.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/types.h>

/* Este programa crea un abanico de procsos. */
/* Ejm 2.6 del libro UNIX Programacion Practica - Kay Robbins */
/* Steve Robbins */

#define N 8

int main(void)
{ int i,status;
  pid_t child,pid_padre;

  pid_padre=getpid();
  for(i=0;i<N; ++i)
    if(!(child=fork())<=0) break;
  if(pid_padre==getpid()) for(i=0;i<N;++i) wait(&status);
  return 0;
}
```

Modifique el código para que el primer proceso creado por el padre, elimine a todos los procesos hermanos, y también al proceso padre, enviando la señal SIGKILL.

ChatGPT respondió con el siguiente código (se encuentra en Paideia como 1.c)

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <sys/wait.h>
5 #include <sys/types.h>
6 #include <signal.h>
7
8 #define N 8
9
10 int main(void) {
11     pid_t pid_padre = getpid();
12     pid_t pid_hijos[N]; // Para guardar los PIDs de los hijos
13     pid_t pid = 0;
14     int i;
15
16     // Creamos una cadena de procesos
17     for (i = 0; i < N; ++i) {
18         pid = fork();
19         if (pid < 0) {
20             perror("fork");
21             exit(1);
22         } else if (pid == 0) {
23             // Estamos en el hijo
24             break;
25         } else {
26             // Proceso padre guarda el PID del hijo
27             pid_hijos[i] = pid;
28
29             // Solo el padre original guarda todos los PIDs
30             if (getpid() != pid_padre) {
31                 break;
32             }
33         }
34     }
35
36     // Primer hijo (creado directamente por el padre)
37     if (i == 0 && getpid() == pid_padre) {
38         sleep(1); // Esperamos que todos los procesos se creen
39
40         // Matamos a los hermanos
41         for (int j = 1; j < N; ++j) {
42             kill(pid_hijos[j], SIGKILL);
43         }
44
45         // Matamos al padre
46         kill(pid_padre, SIGKILL);
47
48         printf("Primer hijo (%d): Eliminé a mis hermanos y al padre (%d)\n", getpid(), pid_padre);
49     }
50
51     pause(); // Cada proceso se queda esperando (para ser eliminado o detenido)
52     return 0;
53 }
54
```

a) (4 puntos) En un archivo texto con nombre *1A.txt* explique porqué este código es incorrecto, es decir no cumple con lo especificado en el problema. Copie el archivo *1.c* con nombre *1A.c* y añada al código líneas que demuestren que el código es incorrecto. Emplee las salidas del código modificado para justificar su explicación. Debe presentar el código modificado *1A.c* y el archivo texto *1A.txt*.

b) (2 puntos) En un archivo con nombre *1B.txt* explique por qué aparentemente el programa se comporta como si verdaderamente cumpliera lo especificado.

c) (4 puntos) Copie el archivo *1.c* con nombre *1C.c* y modifíquelo de forma que resuelva el problema solicitado.

2) (10 puntos) Se tiene un proceso principal que crea un abanico de n procesos hijos. Donde n es pasado como argumento por la línea de ordenes. Si es par le suma 1, de forma que el proceso padre siempre cree un número impar de hijos. Una vez que todos los procesos han sido creados se ejecuta `ps tree`, luego el proceso que se encuentra exactamente en medio, debe eliminar a todos los procesos “hermanos” y por último al proceso padre. Después de cada proceso eliminado se debe ejecutar la orden `ps tree`. De forma que se pueda visualizar cómo el árbol va disminuyendo hasta quedar el proceso medio.

En Paideia se le proporciona el código base: *2.c*

Porf. Alejandro T. Bello Ruiz